Ex.No:8

Date: **Testing of Requirements**

Aim

The objective of this experiment is to analyze the testing efficiency of a program by studying its control flow structure. It focuses on determining the Cyclomatic Complexity and identifying independent execution paths, which helps in designing the minimum number of test cases required for complete coverage.

Problem Description

Software testing requires systematic analysis to ensure that all logical paths of a program are executed. In this experiment, the program is represented using a Control Flow Graph (CFG), where nodes indicate program statements and edges show the flow of execution. By evaluating this representation, testers can understand the complexity of the program and ensure that all possible paths are validated.

Given Conditions

The program under consideration contains basic constructs such as sequence statements, conditional branches, and loops. It has a single starting point and one termination point. Each segment of code is mapped as a node, and transitions between segments are represented as edges. The required parameters include:

- Number of Nodes (N)
- Number of Edges (E)
- Number of Decision Points (P)

These parameters are essential for computing complexity and test coverage.

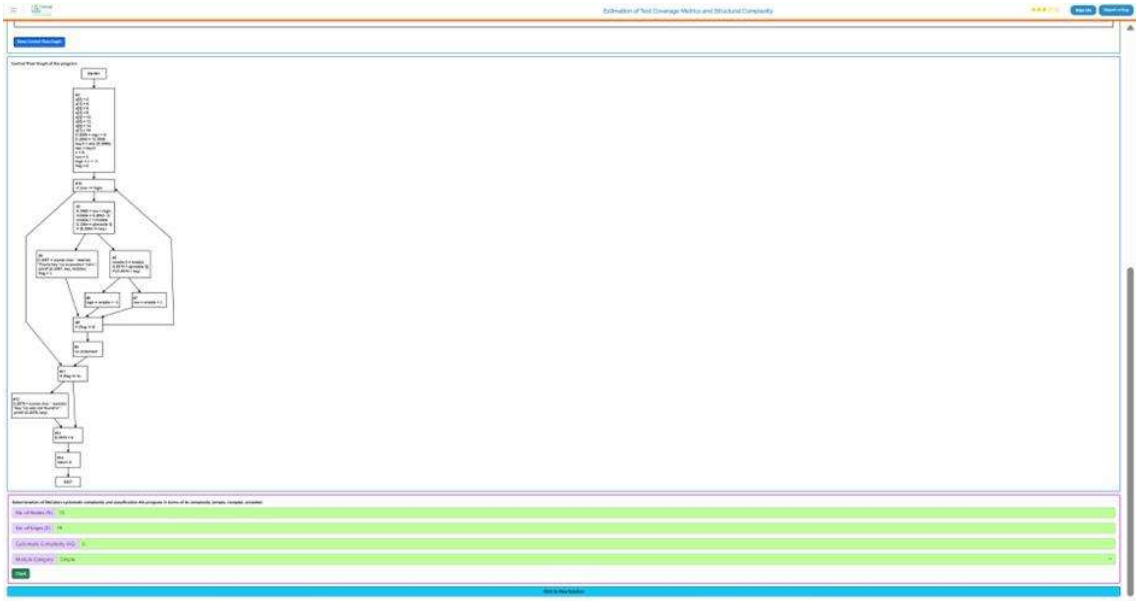
Methodologies

Initially, the program is examined and divided into smaller logical units. These units are represented as nodes in the control flow graph. Connections between nodes are established based on the program's execution sequence. After constructing the graph, the total number of nodes and edges is calculated. Using

these values, Cyclomatic Complexity is determined. Finally, all independent paths are identified, and appropriate test cases are designed to ensure that every path is executed at least once.

About the Node Diagram

The node diagram, or Control Flow Graph, visually represents the execution flow of a program. Each node corresponds to a specific statement or block, and arrows between nodes indicate the order of execution. Conditional and looping structures create branching points in the graph. This diagram is useful for understanding how different parts of the program interact and for identifying all possible execution paths.



Working of Nodes

The working of nodes in a control flow graph involves identifying how each statement connects and contributes to program execution. The complexity of the graph is measured using Cyclomatic Complexity. The formulas used are:

$$V(G) = E - N + 2$$

Where:

- $V(G)$= Cyclomatic Complexity
- E= Number of edges
- N= Number of nodes

Another equivalent formula is:

$$V(G) = P + 1$$

Where:

- P= Number of predicate (decision) nodes

These formulas help determine the number of independent paths in the program. Each independent path represents a unique execution route that must be tested. Nodes work together by forming these paths, ensuring that all logical conditions and loops are properly validated.

Result

After constructing the control flow graph, the values of nodes and edges are obtained. For instance, if N = 10 and E = 11, then:

- $V(G) = 11 - 10 + 2 = 3$

This means there are 3 independent paths in the program. Therefore, a minimum of 3 test cases is required to achieve full path coverage. The result indicates that the program has moderate complexity.

Conclusion

This experiment highlights the importance of Control Flow Graphs and Cyclomatic Complexity in software testing. By analyzing the structure of a program, testers can identify all independent paths and design effective test cases. This approach improves the reliability and quality of software by ensuring thorough testing and reducing the chances of undetected errors.